

Part 3: Writing R Functions

Summary

- This Part covers the programming constructs needed to write R functions.

- First, note how the arguments to the function `whichclosest()` are specified, and that they can be given default values. (In the example, `na.rm` has value `FALSE` by default.)
- Then, note how curly brackets `{ }` are used to wrap the body of the function. Curly brackets are also used to wrap chunks of code, e.g., following an `if()` statement.
- Finally, the `return()` function exits and passes back output. The call to `return()` at the very end of a function is implied and can be omitted, i.e., the final line could have just been:

```
c(zsrt$ix[minpos], zsrt$ix[minpos+1])
```

A Simple Example

- First, we start with a simple example of an R function:

```
> whichclosest = function(z, na.rm=FALSE)
+ {
+   if((!na.rm & any(is.na(z))) | sum(!is.na(z)) <= 1)
+   {
+     return(NA)
+   }
+
+   zsrt = sort(z, index.return=TRUE, na.last=TRUE)
+   minpos = which.min(diff(zsrt$x))
+   return(c(zsrt$ix[minpos], zsrt$ix[minpos+1]))
+ }
```

- Exercise:** Make sure to understand the functions and their usage in `whichclosest()`.

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

12 _____

- 1 _____
- 2 _____
- 3 _____
- 4 _____
- 5 _____
- 6 _____
- 7 _____
- 8 _____
- 9 _____
- 10 _____
- 11 _____
- 12 _____

- 1 **Arguments**
- 2 How function arguments are handled was discussed some in the preced-
- 3 ing Part. Recall that arguments do not need to be provided in a fixed order
- 4 if they are named by the calling procedure.
- 5 Also, recall the useful `args()` function; it works equally well on user-
- 6 defined functions.

- 1 The default arguments to a function can depend on other arguments.
- 2 In this example, the default behavior of `meanT()` is to return the 10%
- 3 trimmed mean if the length of the data vector exceeds 500.

```
> meanT = function(x, trim = (length(x) > 500) * 0.1)
+ {
+   return(mean(x, trim=trim))
+ }
>
> x = rnorm(600)
> meanT(x)
[1] -0.03314024
```

```
> meanT(x, trim=0)
[1] -0.01451877
```

- 1 Another useful feature when working with function arguments is the `...`
- 2 syntax. If `...` is included at the end of an argument list, additional argu-
- 3 ments can be passed into the function, and they will be passed on to other
- 4 functions called within that function.
- 5 Typically this is utilized when the user-defined function calls a sophisti-
- 6 cated function which can potentially have many arguments:

```
> newfunc = function(x, y, ...)
+ {
+   # Some R code here
+   holdlm = lm(y~x, ...)
+   # Some more R code here
+ }
```

1 Lexical Scoping

2 The term **lexical scoping** refers to general rules that R uses to find the values associated with objects.

4 The entire topic can be quite complicated, as R is capable of handling these situations in a sophisticated manner using a range of functions to create and alter the current **environment**.

7 At this stage, it is important to understand how R handles objects referred to within functions. We will do this through a series of simple examples.

1 R looks first within the function to see if an object is defined. If it is not, then it looks into the **parent environment** to see if it has a value.

3 In the example below, `x` is defined to have a value 10 outside of the function `funcex1()`, and this function does not itself define `x`. Hence, `funcex1()` uses `x` equal to 10.

```
> funcex1 = function(y)
+ {
+   return(x+y)
+ }
>
> x = 10
> funcex1(5)
[1] 15
```

- 1 If the function internally assigns a value to `x`, the function will use that.
- 2 But, note that the value of `x` in the parent environment is unchanged.

```
> funcex2 = function(y)
+ {
+   x = 12
+   return(x+y)
+ }
>
> x = 10
> funcex2(5)
[1] 17
```

```
> print(x)
[1] 10
```

- 1 This illustrates that you can overwrite **some** objects (including functions) in the local environment that exist in a parent (including the system level):

```
> T = 0
> T == TRUE
[1] FALSE

> T == FALSE
[1] TRUE
```

- 3 After removing the newly-created object, R will again look into the parent frame for its value:

```
> rm(T)
> T == TRUE
[1] TRUE
```

- 1 Syntax such as the following will not alter the value of `x` in the parent
- 2 environment:

```
> funcex3 = function(y)
+ {
+   x = x + 2
+   return(x+y)
+ }
>
> x = 10
> funcex3(5)
[1] 17
```

```
> print(x)
[1] 10
```

- 1 The `assign()` function does allow for altering objects in the parent envi-
- 2 ronment. Use with `pos = 1`.

```
> funcex4 = function(y)
+ {
+   assign("x", x+2, pos=1)
+   return(x+y)
+ }
>
> x = 10
> funcex4(5)
[1] 17
```

```
> print(x)
[1] 12
```

- 1 **Exercise:** Look at `help(assign)` to understand how the `pos` argument
- 2 works. Look also at `search()`.

3 _____

4 _____

5 _____

6 _____

7 _____

8 _____

9 _____

10 _____

11 _____

12 _____

- 1 The function `do.call()` can also be used in the global environment:

```
> funcex5 = function(trim=0)
+ {
+   xmean = do.call("mean", list(c(x), trim=trim),
+                               env=globalenv())
+   return(xmean)
+ }
>
> x = c(10:20, 1000)
> funcex5(0.1)
[1] 15.5
```

- 1 Things start to get odd when you have an argument to a function whose
- 2 name matches that of an object in the parent frame:

```
> funcex6 = function(x, y)
+ {
+   return(x+y)
+ }
>
> x = 10
> funcex6(x=x, 5)
[1] 15
```

```
> funcex6(x=4, y=x)
[1] 14
```

1 Control Statements

- 2 The flow of the code can be controlled using standard constructs such as
- 3 the if-else statement:

```
> funcex7 = function(x)
+ {
+   if(x > 0)
+   {
+     print("x is positive")
+   } else
+   {
+     print("x is not positive")
+   }
+ }
```

- 1 The ifelse() function gives a compact way of representing the condi-
- 2 tional statement. It allows for vector conditions in a simple way.
- 3 The following example changes all -999 in x into NA:

```
> x = ifelse(x == -999, NA, x)
```

- 1 The for() statement is most often used to loop over integers:

```
> holdlist = list(1:10, 5:20, 40:90)
> for(i in 1:length(holdlist))
+ {
+   print(mean(holdlist[[i]]))
+ }
```

- 2 but it can loop over any vector or list:

```
> for(x in holdlist)
+ {
+   print(length(x))
+ }
```

- 3 The break() and next() functions exit the for() loop and skip to the
- 4 next iteration, respectively.

- 1 **Exercise:** Often `for()` loops can be removed and cleaner code results.
2 Rewrite the code segments above without a `for()` loop.

3 _____
4 _____
5 _____
6 _____
7 _____
8 _____
9 _____
10 _____
11 _____

- 1 It is a myth that `for()` loops are necessarily slower than other approaches
2 in R. Consider the following example:

```
> x = matrix(rnorm(10000000), ncol=100000)
> system.time(apply(x, 2, mean))
      user  system elapsed 
0.456    0.040    0.495
```

```
> meanout = numeric(ncol(x))
> system.time(for(i in 1:ncol(x))
+ {
+   meanout[i] = mean(x[,i])
+ })
      user  system elapsed 
0.336    0.027    0.362
```

- 1 You definitely should use vector arithmetic when possible, however:

```
> x = rnorm(10000000); y = rnorm(10000000)
> z = numeric(10000000)
>
> system.time(for(i in 1:length(z))
+ {
+   z[i] = x[i] + y[i]
+ })
      user  system elapsed 
0.701    0.020    0.721

> system.time(assign("z", x+y))
      user  system elapsed 
0.017    0.000    0.016
```

- 1 **Exercise:** Suppose that `meanvec` is a numeric vector. The goal is create
2 a vector of psuedo-random variates drawn from the normal distribution
3 with the mean of each draw given by the corresponding entry in `meanvec`.
4 The variance is always one. Compare the time it takes to do this with and
5 without a `for()` loop.

6 _____
7 _____
8 _____
9 _____
10 _____
11 _____

- 1 The `while()` function continues a loop until the given condition is false.
- 2 This example waits for a random sample with mean of at least 0.5:

```
> x = rnorm(100); nsamp = 1
> while(mean(x) < 0.5)
+ {
+   x = rnorm(100)
+   nsamp = nsamp + 1
+ }
> mean(x)
[1] 0.527035

> print(nsamp)
[1] 174920
```

1 Returning Information

- 2 As seen above, the `return()` call exits the function. There can be multiple
 - 3 calls to `return()` in any one function.
 - 4 The call
- ```
> return()
```
- 5 can be used to exit a function only returning the special symbol `NULL`. This
  - 6 can be useful if, for example, the function is only intended to generate a
  - 7 plot or file.
  - 8 It is very common to see `return()` used along with `list()` to enable the
  - 9 function to return different types of information.

- 1 The `stop()` function can be used to exit a function with an error condition.
- 2 It is a good idea to do this instead of using `return()` since R has special
- 3 functions for handling errors. See also `warning()` for passing back warn-
- 4 ings without exiting the function.

```
> funcex7 = function(x)
+ {
+ if(!is.numeric(x))
+ {
+ stop("Input argument must be numeric")
+ }
+ return(x^2)
+ }
```

### 1 Infix Operators

- 2 R also allows you to create **infix operators**. Consider the following exam-
- 3 ple:

```
> "%_%" = function(str1, str2)
+ {
+ paste(str1, str2, sep="_")
+ }
> "filename" %_% 1 %_% 12
[1] "filename_1_12"
```

- 4 Note that the operator must start and end with `%`.

1 **Exercise:** Create a new addition operator `%+%` that returns `x` when given `x`  
2 `%+%` `NA`. (Note that the default addition operator would return `NA` in that  
3 case.) The new operator should still return `NA` when passed `NA %+%` `NA`.

4 \_\_\_\_\_

5 \_\_\_\_\_

6 \_\_\_\_\_

7 \_\_\_\_\_

8 \_\_\_\_\_

9 \_\_\_\_\_

10 \_\_\_\_\_

11 \_\_\_\_\_

12